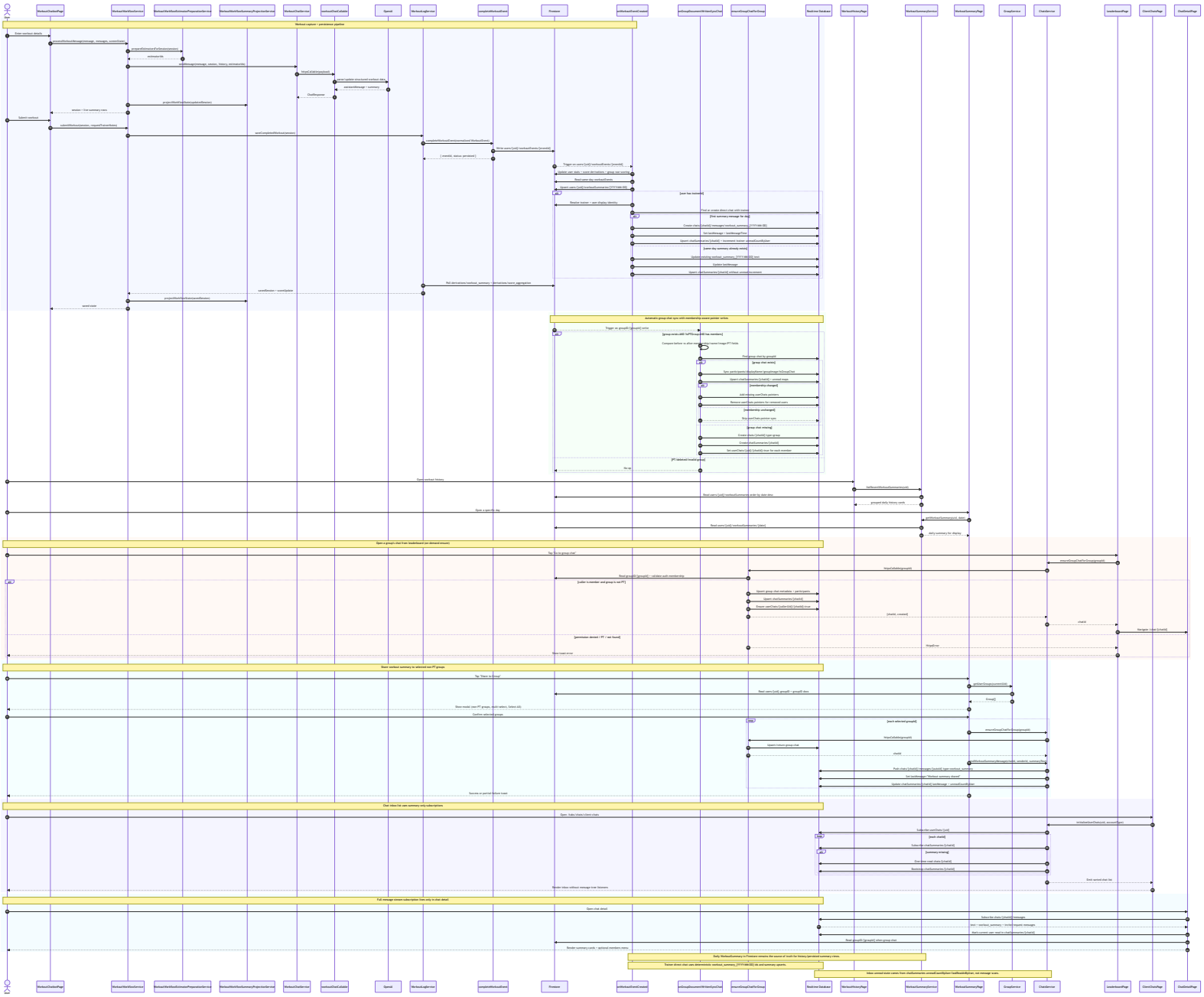
User
WorkoutChatterPage
WorkoutWorkflowService
WorkoutWorkflowSessionPreparationService
WorkoutWorkflowSummaryProjectionService
WorkoutChatService
semiosisChatCallable
Openai
WorkoutLogService
completeWorkoutEvent
Firestore
setWorkoutEventCreated
onGroupDocumentWritten(CreateChat)
ensureGroupChatFor(Group)
Realtime Database
WorkoutHistoryPage
WorkoutSummaryservice
WorkoutSummaryPage
Groupservice
ChatsService
LeaderboardPage
ClientChatsPage
ChatDetailPage
Workout capture + pre-answer pipeline
Enter workout details
startWorkoutMessage(message, messages, sessionCache)
prepareWorkoutsFor(Sessions|sessions)
exercise-lib
sendmessage(message, session, history, exercise-lib)
fetchCoachHistory(uid)
parse+update structured workout data
narrateWorkoutMessage + summary
ChatResponse
projectWorkoutFrom(updatedSessions)
session + live summary rows
Submit workout
submitWorkout(session, requestTrainerReview)
saveCompletedWorkout(session)
normalizeWorkoutEvents(storedUserWorkoutEvents)
Write users/{uid}/workoutEvents/{eventId}
{ eventId, status: persisted }
Trigger on users/{uid}/workoutEvents/{eventId}
Update user stats + assist derivatives + group user rosters
Read owner-day workoutEvents
Upsert users/{uid}/workoutSummaries/{YYYY-MM-DD}
[user has trainer]
Resolve trainer + user display identity
Find or create direct chat with trainer
[first summary message for day]
Create chats/{chatId}/messages/workout_summary/{YYYY-MM-DD}
Set lastMessage + lastMessageTime
Upsert chatSummaries/{chatId} + increment trainer unreadCountByUser
[trainer-day summary already exists]
Update existing workout_summary/{YYYY-MM-DD} text
Update lastMessage
Upsert chatSummaries/{chatId} without unreadIncrement
Poll derivatives/workout_summary + derivatives/xxxx_aggregation
saveSession + scoreUpdate
projectWorkoutFrom(updatedSessions)
saved state
Automatic group chat sync with membership-aware pointer setting
Trigger on groups/{|groupId} write
[group exists AND hasPCGroupAND has members]
Compare before/ to after/ group name/image/PT fields
Find/ensure chat by groupId
[group chat exists]
Sync participants/displayNames/groupImage toGroupChat
Upsert chatSummaries/{chatId} + unread maps
[membership changed]
Add missing userChats pointers
Remove userChats pointers for removed users
[membership unchanged]
Stale userChats pointer sync
[group chat missing]
Create chats/{chatId} type=group
Create chatSummaries/{chatId}
Set userChats/{uid}/{chatId}=true for each member
[PT-labeled trainer+group]
No-op
Open workout history
isReferenceWorkoutSummaries(uid)
Read users/{uid}/workoutSummaries order by date desc
grouped daily history cards
Open a specific day
getWorkoutSummary(uid, date)
Read users/{uid}/workoutSummaries/{date}
daily summary for display
Open a group's chat from leaderboard (on demand ensure)
Tap "Go to group chat"
ensureGroupChatFor(Group|groupId)
hitpsCallable(groupId)
Read groups/{|groupId} + validate auth+membership
[caller is member and group is not PT]
Upsert group chat metadata + participants
Upsert chatSummaries/{chatId}
Ensure userChats/{callerUid}/{chatId}=true
{chatId, created}
chatId
Navigate /chat/{chatId}
[permission denied / PT / not found]
HttpsError
Show toast/error
Share workout summary to selected non-PT groups
Tap "Share to Group"
getUserGroups(currentUid)
Read users/{uid}/ groups/{0 – groups/9} docs
Groups[]
Show modal (non-PT groups, multi-select, Select-All)
Confirm selected groups
[each selected group|dd]
ensureGroupChatFor(Group|groupId)
hitpsCallable(groupId)
Upsert/ensure group chat
chatId
postWorkoutSummaryMessage(chatId, senderUid, summaryText)
Push chats/{chatId}/messages/{autoId} type=workout_summary
Set lastMessage + lastMessageAt
Update chatSummaries/{chatId} lastMessage + unreadCountByUser
Success or partial failure toast
Chat inbox list uses summary-only subscriptions
Open /inbox/chats/client-chats
ensureUserChats(uid, accountRoles)
Subscribe userChats/{uid}
[each chatId]
Subscribe chatSummaries/{chatId}
[summary missing]
Ensure direct+trial chats
Resolve/ensure chatSummaries/{chatId}
Dock opened chat dot
Render inbox without message-tree listeners
Full message-stream subscription (one-only in chat detail)
Open chat detail
Subscribe chats/{chatId}/messages
limit + startAfter(_summary + limited recent messages
Batch current user read in chatSummaries/{chatId}
Read groups/{|groupId} when group chat
Render summary cards + optional members menu
Daily WorkoutSummary in Firestore remains the source of truth for history/persisted summary views.
Trainer direct chat uses deterministic workout_summary/{YYYY-MM-DD} id and summary upserts.
Inbox unread state comes from chatSummaries.unreadCountByUser/lastReadAtByUser, not message scans.